The Folding Tape: Version-Locked, Context-Aware Help Systems

for Free and Open Source Software via
Telemetry-Grounded Retrieval-Augmented Generation
and Hierarchical Memory Compression

Daniel Scott Matthews
Independent Researcher

April 2026

Abstract

We propose a novel architecture for embedded, context-aware help systems in free and open source software (FOSS) applications. The system comprises three tightly coupled components that ship as part of the standard release artefact: a version-locked vector database of project documentation and source-derived knowledge, a quantised small language model for local inference, and a structured telemetry log that supplies UI interaction context to augment retrieval queries. Together these components produce a help interface that is accurate to the exact release version, fully offline, privacy-preserving by construction, and semantically sensitive to what the user is currently doing. A central contribution is the *folding tape* memory model, a hierarchical compression scheme applied progressively to the telemetry stream, in which the representation of past events becomes increasingly abstract and compact as those events recede in time. This mirrors the structure of human working memory, and produces context vectors that are simultaneously token-efficient and semantically richer than raw event logs. We argue that this architecture represents a practically achievable and currently underexplored alternative to cloud-dependent AI help systems, suitable for air-gapped deployments, privacy-sensitive environments, and the broad ecosystem of FOSS applications.

Keywords: retrieval-augmented generation, small language models, context-aware help, telemetry, memory compression, FOSS, local inference, human-computer interaction

Contents

1 Introduction **4**

2 Background and Related Work **5**
2.1 Retrieval-Augmented Generation . . . 5
2.2 Small Language Models for Local Inference . . . 5
2.3 Context-Aware Help in Software Systems . . . 5
2.4 Memory Compression in Language Models . . . 6

3 System Architecture **6**

4 The Version-Locked Knowledge Base **7**
4.1 Source Material . . . 7
4.2 Chunking Strategy . . . 8
4.3 Embedding and Indexing . . . 8
4.4 Synthetic Question–Answer Pair Generation . . . 9

5 Telemetry as Context **9**
5.1 The Case for Structured Local Telemetry . . . 9
5.2 Event Schema . . . 9
5.3 Error Event Prioritisation . . . 10

6 The Folding Tape Memory Model **10**
6.1 Motivation . . . 10
6.2 Structure . . . 11
6.3 The Compression Pass . . . 11
6.4 Trigger Conditions . . . 13
6.5 What Survives Compression . . . 13
6.6 Relationship to Human Working Memory . . . 13

7 Retrieval-Augmented Generation Pipeline **14**
7.1 Query Construction . . . 14
7.2 Retrieval . . . 14
7.3 Answer Generation . . . 14
7.4 Proactive Surface . . . 15

8 Deployment and Release Engineering **16**
8.1 Release Artefact Structure . . . 16
8.2 CI/CD Integration . . . 16
8.3 Model and Index Versioning . . . 17
8.4 Model Size and Acceptance . . . 17

9 Discussion **17**
9.1 Telemetry and Privacy 17
9.2 Feedback Loop and Documentation Quality 18
9.3 The Context Prioritisation Problem 18
9.4 Limitations 19

10 Conclusion **19**

1 Introduction

The dominant model of AI-assisted software help in 2024–2026 is cloud-centric: a large language model hosted on remote infrastructure receives a user query, generates a response, and returns it over a network connection. While capable, this model carries a cluster of properties that are poorly suited to a significant segment of the software ecosystem: it requires persistent internet access, it raises legitimate privacy concerns about what queries are transmitted to whom, it decouples the knowledge available to the model from the specific version of the software the user is running, and it creates an external runtime dependency for what ought to be a local, always-available feature.

Free and open source software in particular is routinely deployed in contexts where these constraints are acute: air-gapped enterprise networks, embedded and industrial systems, security-sensitive environments, and geographies with limited or unreliable connectivity. More broadly, FOSS users and developers tend to have higher expectations around privacy, auditability, and self-determination than users of proprietary software. A help system that phones home is, for many such users, simply not acceptable regardless of its capability.

At the same time, the help systems that ship with most FOSS applications remain in a pre-AI state: static HTML pages, PDF manuals, man pages, or at best full-text search over a documentation corpus. These systems are accurate and offline, but they are not semantic, not contextual, and not conversational. The user must already know how to phrase their question in terms the documentation uses, and they receive no assistance that reflects what they were actually doing when they got stuck.

This paper proposes a system that captures the advantages of both approaches while inheriting the drawbacks of neither. The core insight is that all three necessary components — a curated knowledge base, a language model capable of synthesising answers, and a source of UI context — can be made local, version-locked, and shippable as ordinary release artefacts. The knowledge base takes the form of a vector database generated from the project's own source code and documentation at release time. The language model is a quantised small model running via CPU-compatible inference. The context is derived from the application's own structured telemetry log, which records UI interaction events locally and never leaves the user's machine.

The novel contribution that ties these components together is the *folding tape*: a hierarchical compression model for the telemetry stream in which older events are progressively abstracted into compact, high-level semantic tokens. Rather than maintaining a sliding window that discards history at a fixed horizon, or a flat log that grows without bound, the folding tape maintains a complete representation of the session at monotonically decreasing resolution. The oldest events are represented by a handful of abstract tokens capturing their essential meaning; the most recent events are preserved at full fidelity. This structure is computationally efficient, mirrors the architecture of human episodic memory, and produces context vectors that are directly useful for retrieval augmentation without requiring the model to infer intent from raw event sequences.

The remainder of this paper is organised as follows. Section 2 reviews related work. Section 3 describes the overall system architecture. Section 4 covers the construction of the version-locked knowledge base. Section 5 describes the telemetry collection and structuring layer. Section 6 presents the folding tape memory model in detail. Section 7 describes the

retrieval-augmented generation pipeline. Section 8 addresses deployment and release engineering considerations. Section 9 discusses open problems, limitations, and future directions. Section 10 concludes.

2 Background and Related Work

2.1 Retrieval-Augmented Generation

Retrieval-augmented generation (RAG) [1] combines a dense retrieval component with a generative language model. At inference time, a query is encoded into a vector embedding, and semantically similar passages are retrieved from a pre-built vector index. The retrieved passages are included in the model's context window alongside the query, grounding the generated response in specific source material rather than requiring the model to recall facts from its parameters alone.

RAG has become the dominant approach for knowledge-grounded question answering in production systems, offering several advantages over pure parametric approaches: knowledge can be updated without retraining, retrieval provides a degree of interpretability (the user can see which sources were consulted), and the model's context window constrains the answer to retrieved content, reducing hallucination. For our purposes, the most important property of RAG is that the knowledge base is a separable artefact — a vector index file that can be generated at build time and shipped with the application, independently of the model weights.

2.2 Small Language Models for Local Inference

The landscape of small language models (SLMs) capable of local CPU inference has changed substantially between 2023 and 2026. Models in the 1–3 billion parameter range, quantised to 4-bit or 8-bit integer precision using schemes such as GPTQ [2] or GGUF [3], achieve acceptable response latency on commodity hardware without GPU acceleration. Inference engines such as `llama.cpp` [3] expose these models through a C API with Python and Rust bindings, making integration into native desktop applications straightforward.

Models such as Phi-3-Mini [4], Qwen2.5-1.5B [5], and Gemma-2B [6] have demonstrated strong performance on instruction-following and question-answering benchmarks at parameter counts that produce GGUF files of 1–2 GB — within the range of what users routinely tolerate as part of a software package, particularly for professional tools.

2.3 Context-Aware Help in Software Systems

The problem of making software help contextually aware has a long history. Early work on adaptive user interfaces [7] explored systems that would modify their behaviour based on observed user actions. Intelligent tutoring systems [8] used detailed models of user knowledge state to select appropriate instructional content. More recently, Microsoft's Copilot and Apple Intelligence represent OS-level attempts to make AI assistance aware of the current application state, though both rely on cloud inference and proprietary accessibility APIs.

The closest prior work to our proposal is the concept of context-sensitive help as standardised in frameworks such as Qt's QWhatsThis and the JavaHelp API, which surface documentation relevant to the currently focused widget. These systems are syntactic rather than semantic — they retrieve by exact widget identifier rather than by meaning — and they have no model of the user's recent history or intent.

2.4 Memory Compression in Language Models

The challenge of maintaining useful representations of long interaction histories within the finite context window of a language model has attracted significant attention. Approaches include hierarchical summarisation [9], memory-augmented transformers [10], and recurrent compression mechanisms [11]. The MemGPT architecture [12] introduced explicit memory management where a controller decides what to move between in-context and external memory stores.

Our folding tape model differs from these approaches in several respects. It is not applied to conversation history between the user and the model, but to a structured telemetry stream of UI events. The compression passes are themselves model inference steps, but they may be performed asynchronously in the background by a smaller, faster summarisation model. And crucially, the compression is not lossless: it is designed to be semantically lossy in a specific way, discarding low-level operational detail while retaining high-level intent and failure signals.

3 System Architecture

Figure 1 shows the overall architecture. The system divides cleanly into a *build-time pipeline*, which executes once per release and produces two static artefacts, and a *runtime pipeline*, which runs continuously within the application process.

The two build-time artefacts are:

- `help.db`: a vector database, implemented as a single file using a library such as `sqlite-vec` or `usearch`, containing dense embeddings of chunked documentation and source-derived knowledge.
- `help.model`: a quantised small language model in GGUF or ONNX format, capable of CPU inference via `llama.cpp` or ONNX Runtime.

These artefacts are included in the release package alongside the application binary, the conventional documentation, and the project licence. They are versioned identically to the release. A user running version 2.4.1 of the application has `help.db` and `help.model` that correspond exactly and exclusively to the knowledge available in version 2.4.1.

The runtime pipeline consists of three stages: telemetry collection and folding (Section 5 and 6), retrieval augmentation (Section 7), and answer generation. The user may explicitly invoke the help system by opening a help panel or pressing a designated key, or the system may surface relevant content proactively based on detected workflow state (e.g., repeated failed attempts at an operation).

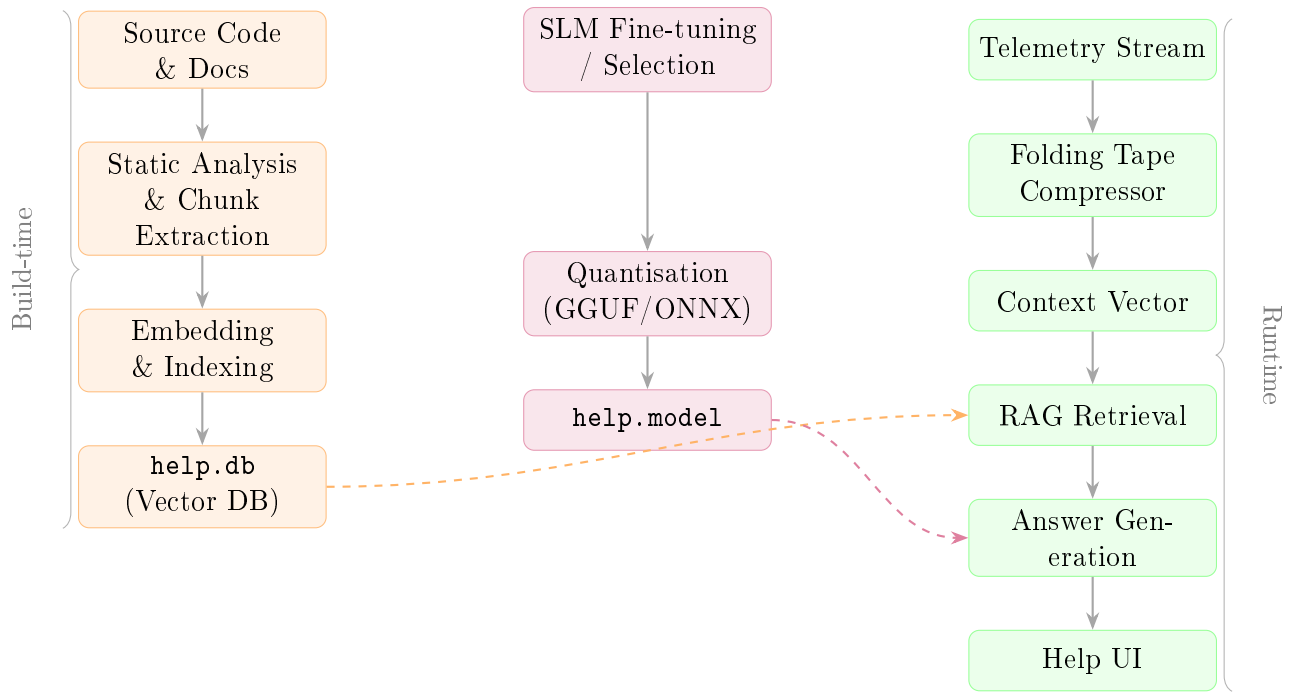

Figure 1: Overall system architecture. The left and centre columns are executed once at release build time, producing `help.db` and `help.model` as static artefacts. The right column executes at runtime within the application process. Dashed arrows indicate the static artefacts being loaded at runtime.

4 The Version-Locked Knowledge Base

4.1 Source Material

For a FOSS project, the complete knowledge needed to answer user questions is, in principle, available in the repository. The build pipeline extracts knowledge from the following sources, roughly in descending order of reliability:

Docstrings and inline documentation. Functions, classes, methods, and modules documented with docstrings constitute high-quality, authoritative knowledge. The signature establishes the contract; the docstring explains intent, parameters, return values, and exceptions.

User-facing documentation. Markdown, RST, or AsciiDoc files in `docs/` directories, wiki content, man pages, and README files constitute the project's intentional description of itself to its users. This is the most direct source for answering "how do I" questions.

Test suites. Tests are executable specifications. A test that invokes a function with particular arguments and asserts a particular output is a precise, verified usage example. Extract-

ing test cases and reformatting them as examples is a reliable way to populate the knowledge base with demonstrably correct code.

Configuration schemas and CLI definitions. Argument parsers, configuration file schemas, and command-line interface definitions enumerate all user-facing options, their types, defaults, and constraints. These are often the most complete and up-to-date source of truth for configuration-related questions.

Commit messages and changelogs. The history of *why* things changed is not available from the current source snapshot alone. Commit messages, particularly those tagged as fixes or behaviour changes, carry important knowledge about edge cases, deprecated behaviours, and the rationale for current design decisions.

4.2 Chunking Strategy

The knowledge base is not indexed as raw files but as semantically coherent chunks. The chunking strategy balances two competing requirements: chunks should be small enough that retrieved passages are specific and relevant, but large enough that they are self-contained and interpretable without surrounding context.

We adopt a hierarchical chunking approach:

1. **Structural chunks**: docstrings with their associated signatures, individual test cases, individual documentation sections. These are the primary unit.

2. **Contextual enrichment**: each chunk is augmented with a short prefix indicating its source type, module path, and version tag. This metadata participates in the embedding and is returned with retrieved chunks to help the generation model contextualise its response.

3. **Overlap**: adjacent structural chunks share a small token overlap to avoid context being lost at boundaries.

4.3 Embedding and Indexing

Chunks are embedded using a small, efficient sentence embedding model — such as `all-MiniLM-L6-v2` or `bge-small-en` — that can be run as a batch process at build time without GPU acceleration in acceptable wall-clock time. The resulting vectors are stored in a single-file vector index. We recommend `sqlite-vec` for projects prioritising minimal dependencies, as it extends SQLite with vector operations and produces a standard `.db` file with no additional runtime requirements. For projects prioritising retrieval speed at scale, `usearch` provides a more performant flat or HNSW index.

The build pipeline is fully automatable and integrates naturally into existing CI/CD workflows. It requires no manual curation beyond the existing project documentation. For a mid-sized FOSS project with ~100k tokens of documentation and docstrings, the pipeline completes in under ten minutes on standard CI infrastructure.

4.4 Synthetic Question–Answer Pair Generation

Optionally, the build pipeline can use a large external model (at build time only, not at runtime) to generate synthetic question–answer pairs from the chunked knowledge base. These pairs serve two purposes: they provide a fine-tuning dataset for the bundled small model, and they serve as an evaluation set for measuring retrieval and generation quality across releases.

The generation prompt instructs the model to produce questions that a genuine user would ask, grounded in the content of each chunk, along with reference answers that cite only the information in the chunk. The result is a structured dataset that captures the question–answer character of the target task, suitable for supervised fine-tuning of the SLM component.

5 Telemetry as Context

5.1 The Case for Structured Local Telemetry

Most production applications, including many FOSS ones, already maintain some form of event log for debugging, crash reporting, or usage analytics. We propose that this infrastructure, when oriented towards local storage with user-auditable semantics, constitutes the ideal context source for a help system.

The key properties of structured telemetry that make it superior to alternatives — such as live accessibility tree inspection or screen vision models — are:

Structure. Telemetry events are typed and named. An event `tool.selected{name: "UV Unwrap"}` is unambiguous in a way that a screenshot or accessibility tree node is not. The application already knows what it is doing; telemetry simply records it.

Causal ordering. The log records events in the order they occurred, preserving the causal chain that led to the current state. Context is not just "what is on screen now" but "what the user has been trying to do and where they got stuck."

Privacy by construction. A local telemetry log that never leaves the machine is privacy-preserving by architecture, not by policy. Users of FOSS applications can audit the log format, inspect the log file, and opt out of collection without any network-level trust being required.

Existing infrastructure. For projects that already instrument their codebase with telemetry for debugging or analytics, the incremental cost of producing a help-oriented context stream is low.

5.2 Event Schema

The telemetry stream is a sequence of typed events, each carrying a timestamp, an event type, and a payload of key–value pairs. A minimal schema for a graphics or productivity application might include:

Listing 1: Example telemetry event schema

```python
@dataclass
class TelemetryEvent:
    timestamp: float          # Unix epoch, millisecond precision
    event_type: str           # Dotted namespace: "tool.selected"
    payload: dict[str, Any]   # Event-specific fields
    session_id: str           # Stable within an application session
    sequence: int             # Monotonically increasing within session

# Common event types
TOOL_SELECTED    = "tool.selected"        # payload: {name, mode}
OPERATION_APPLIED = "operation.applied"  # payload: {op, result, error?}
PANEL_OPENED     = "panel.opened"        # payload: {panel_id}
ERROR_RAISED     = "error.raised"        # payload: {code, message, context}
HELP_OPENED      = "help.opened"         # payload: {trigger: "explicit"|"proactive"}
TASK_COMPLETED   = "task.completed"      # payload: {task_type, duration_ms}
```

5.3 Error Event Prioritisation

Error events carry disproportionately high contextual value for help purposes. A user who opens help immediately after an error almost certainly wants to understand and resolve that error. Error events are therefore treated specially in the context pipeline:

- Error events within a configurable recency window (default: the last five minutes) are always included in the context vector at full fidelity, regardless of their position in the folding tape.
- Repeated error events of the same type are compacted into a single annotated entry with a repetition count (`error:uv-unwrap:no-seams`$\times$`3`), which itself is a strong signal of user frustration and targeted information need.
- The error payload, including the error code and any contextual fields (current selection, active mode, parameter values), is preserved verbatim and included in the retrieval query.

6 The Folding Tape Memory Model

6.1 Motivation

A naive approach to incorporating telemetry context into a retrieval query would be to include the last N events as a raw prefix. This sliding window approach has two significant

deficiencies. First, it imposes a hard cutoff, discarding all information about the session before position N regardless of its relevance. A user who completed a complex setup procedure thirty minutes ago and is now stuck on a downstream operation has context that is invisible to a sliding window but highly relevant to the help system. Second, raw event sequences are not in the form most useful to a retrieval model: they require inference to determine what the user was *trying* to do, rather than providing that intent directly.

We propose the *folding tape* as an alternative. The folding tape is a data structure that maintains a complete representation of the session history at monotonically decreasing resolution as a function of temporal distance from the present moment.

6.2 Structure

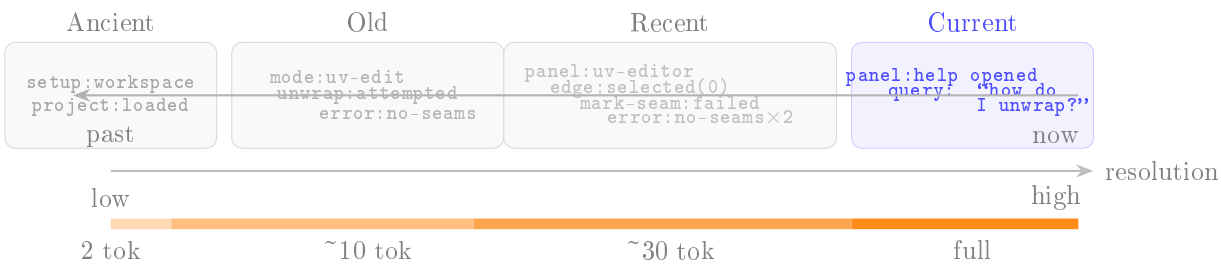

Figure 2: The folding tape structure. Time flows from right (present) to left (past). Events at full resolution are maintained nearest to the present. Successive compression passes collapse older events into progressively more abstract and token-efficient representations, while retaining a trace of the entire session history.

The tape is divided into temporal zones (Figure 2). The rightmost zone (most recent) contains events at full fidelity. Successive zones to the left contain events that have been passed through one, two, or more compression passes respectively. Each compression pass is a summarisation inference that takes a window of events and produces a shorter, more abstract representation.

Formally, let $\mathcal{E} = (e_1, e_2, \ldots, e_n)$ denote the sequence of telemetry events in a session, ordered by timestamp. The folding tape $\mathcal{T}$ is a sequence of zones $\mathcal{T} = (Z_k, Z_{k-1}, \ldots, Z_1, Z_0)$ where Z_0 is the current zone at full fidelity and Z_i for $i > 0$ is the zone i compression passes removed from the raw events. Each zone Z_i contains a set of representations R_i such that:

$$|R_i| \ll |R_{i-1}| \quad \text{and} \quad \text{tokens}(R_i) \ll \text{tokens}(R_{i-1}) \tag{1}$$

where $|\cdot|$ denotes cardinality and $\text{tokens}(\cdot)$ denotes the total token count of the zone's content.

6.3 The Compression Pass

Each compression pass is a summarisation operation applied to a contiguous window of events from the adjacent finer-grained zone. The pass is implemented as an inference call to

a small, fast summarisation model (which may be the same model used for answer generation, or a smaller dedicated model). The prompt instructs the model to:

1. Identify the high-level intent of the event sequence (what was the user trying to accomplish?).
2. Note any failures or errors and their apparent causes.
3. Note any completed tasks or successful transitions.
4. Discard low-level navigational and operational detail that did not contribute to successes or failures.
5. Return the result as a compact sequence of typed abstract tokens.

The output format is deliberately constrained to a short sequence of structured tokens rather than free prose, ensuring predictable token budgets and enabling the output to serve directly as retrieval query terms.

Listing 2: Example compression pass: input and output

```
# Input: raw events from Zone 0 (10 minutes of interaction)
raw_events = [
    "t-600: file.opened{name: project.blend}",
    "t-590: workspace.switched{to: UV Editing}",
    "t-580: tool.selected{name: UV Unwrap}",
    "t-570: operation.applied{op: unwrap, result: error, "
            "error: NoSeamsMarked}",
    "t-560: panel.opened{id: uv-editor}",
    "t-550: mesh.entered{mode: edit}",
    "t-540: selection.changed{edges: 0}",
    "t-530: tool.selected{name: Mark Seam}",
    "t-520: operation.applied{op: mark-seam, result: error, "
            "error: NothingSelected}",
    "t-510: operation.applied{op: mark-seam, result: error, "
            "error: NothingSelected}",
]

# Output: Zone 1 compressed representation (~12 tokens)
compressed = [
    "workflow:uv-unwrap",
    "error:NoSeamsMarked",
    "attempted:mark-seam",
    "error:NothingSelected x2",
    "status:blocked"
]
```

6.4 Trigger Conditions

A compression pass is triggered by any of the following conditions:

- **Time elapsed**: the oldest events in the current zone exceed a configurable age threshold (e.g., 5 minutes for Zone 0 → Zone 1, 20 minutes for Zone 1 → Zone 2).
- **Task boundary events**: the telemetry stream emits a `task.completed` event, indicating a natural transition between workflow phases.
- **Token budget pressure**: the total token count of the context vector approaches a configurable limit, triggering an early compression pass on the oldest zone.

Compression passes are performed asynchronously in a background thread, so they do not block the UI and are not on the critical path when the user opens help.

6.5 What Survives Compression

The compression is intentionally lossy, but the information that is preserved versus discarded is not random — it reflects the semantic importance of events for the purpose of help retrieval.

Survives compression:

- Failures and errors, with their type and context
- Repeated attempts (the repetition itself is a signal)
- Completed task boundaries ("setup complete", "export successful")
- Major mode or context transitions ("switched to UV editing mode")
- High-level workflow intent

Discarded by compression:

- Routine navigation between panels that led to no outcome
- Exact parameter values that did not cause errors
- Successful routine operations that were not task-boundary events
- Fine-grained selection and cursor operations

6.6 Relationship to Human Working Memory

The folding tape structure is isomorphic to the structure of human episodic memory as described in cognitive neuroscience. The hippocampus maintains highly detailed representations of recent events, while older memories are progressively reconsolidated into the neocortex at increasing levels of abstraction, with only semantically significant content surviving the reconsolidation process [13, 14]. The "gist" of an experience — what was attempted,

whether it succeeded, what the outcome was — survives reconsolidation far better than low-level perceptual and operational details.

This parallel is not merely aesthetic. It suggests that the folding tape's compression scheme is well-calibrated to the actual information needs of the help query, because the same compression that human memory applies naturally when a user reflects on what they were trying to do is approximated by the folding tape's summarisation passes. The compressed context vector represents approximately what the user themselves would say if asked to describe their recent experience.

7 Retrieval-Augmented Generation Pipeline

7.1 Query Construction

When the user triggers help — explicitly by opening the help panel, or implicitly by the system detecting a stuck state (e.g., the same error raised three times within a session) — a retrieval query is constructed from two sources:

1. **Explicit user query**: if the user has typed a question, it is included verbatim.
2. **Context vector from the folding tape**: the full folding tape, serialised in order from oldest (most compressed) to most recent (full fidelity), is prepended to the query. In the absence of an explicit user query, the context vector alone is sufficient to drive retrieval.

The combined query is encoded using the same embedding model used at index time. The embedding model was chosen to produce vector representations that capture semantic meaning in a domain-agnostic way, so the context tokens from the folding tape (`workflow:uv-unwrap`, `error:NoSeamsMarked`) naturally map onto the same embedding space as the documentation text that discusses UV unwrapping and seam marking.

7.2 Retrieval

The query embedding is used to retrieve the top-k most semantically similar chunks from `help.db`, using approximate nearest-neighbour search. The retrieved chunks include their metadata: source type (docstring, tutorial, test case, etc.), module or section path, and version tag.

A lightweight reranking step filters the retrieved chunks for relevance to the specific error or workflow state identified in the folding tape context. This reranker is a simple cross-encoder or BM25 hybrid, not a full model inference, and adds negligible latency.

7.3 Answer Generation

The retrieved chunks and the folding tape context are assembled into a prompt for the bundled small language model:

Listing 3: Answer generation prompt structure

```
System: You are a help assistant for [AppName] v[VERSION].
Answer questions using ONLY the documentation provided.
If you cannot answer from the provided documentation, say so.
Do not invent functionality or parameters.

Context (user's recent session):
  [FOLDING TAPE OUTPUT - oldest to newest]

Retrieved documentation:
  [CHUNK 1: source, section, content]
  [CHUNK 2: source, section, content]
  ...

User question: [EXPLICIT QUERY OR INFERRED FROM CONTEXT]

Answer:
```

The system prompt's instruction to answer only from provided documentation is critical for a help system. Users are relying on the system to describe actual software behaviour. Hallucinated API parameters or workflow steps would be actively harmful. The RAG architecture, combined with a strong system prompt constraint, substantially reduces the hallucination risk relative to pure parametric generation, because the model's answer is anchored to specific retrieved text.

7.4 Proactive Surface

The help system need not wait to be explicitly invoked. The folding tape is continuously updated, and the system can monitor it for patterns that reliably predict a user information need:

- The same error raised more than twice in succession
- A known "stuck" pattern: opening and closing the same panel repeatedly without completing a workflow
- An operation attempted immediately after help was closed without completing the task

In these cases, the system can surface a contextually relevant help suggestion in a non-intrusive banner or tooltip, without waiting for the user to explicitly ask. The suggestion is generated using the same RAG pipeline, with the folding tape context as the sole query driver.

8 Deployment and Release Engineering

8.1 Release Artefact Structure

The release package for a FOSS application adopting this architecture contains the following additions relative to a conventional release:

Listing 4: Annotated release package structure

```
myapp-2.4.1/
  bin/myapp                # Application binary (unchanged)
  lib/                     # Application libraries (unchanged)
  share/
    doc/myapp/             # Conventional HTML/man docs (unchanged)
    help/
      help.db              # Vector index: ~50-200 MB depending on docs
      help.model           # Quantised SLM: ~800 MB - 2 GB
      help.embed           # Embedding model weights: ~90 MB
      MANIFEST.json        # Version tags, checksums, model provenance
  LICENSE                  # Project licence (unchanged)
```

The `MANIFEST.json` records the exact version of the project, the embedding model and its provenance, the SLM and its quantisation parameters, and SHA-256 checksums of all help artefacts. This makes the help system's knowledge provenance as auditable as the source code itself.

8.2 CI/CD Integration

The build pipeline that generates `help.db` integrates naturally into existing CI/CD workflows:

Algorithm 1 Help artefact build pipeline

Require: Source tree at release tag v

1: Extract docstrings, documentation files, test cases, CLI definitions
2: Chunk extracted content with overlap and metadata enrichment
3: Embed all chunks using embedding model $\mathcal{M}_e$
4: Write embeddings and metadata to `help.db`
5: (Optional) Generate synthetic Q&A pairs using external LLM
6: (Optional) Fine-tune SLM $\mathcal{M}_s$ on synthetic Q&A pairs
7: Quantise $\mathcal{M}_s$ to GGUF Q4_K_M
8: Compute checksums of `help.db`, `help.model`, `help.embed`
9: Write `MANIFEST.json`
10: Package all help artefacts with release

Steps 5–6 (synthetic Q&A generation and fine-tuning) require access to a large external model and a GPU, and may be omitted for projects that prefer to use a pre-trained SLM

without fine-tuning. In this case, a general-purpose instruction-following SLM is used with the strong system prompt constraint described in Section 7, and the RAG retrieval does the heavy lifting of grounding the response in project-specific knowledge.

8.3 Model and Index Versioning

Because both artefacts are included in the release and tagged with the release version, users who maintain multiple installed versions of an application (common in enterprise and scientific computing environments) automatically have access to the correct help system for each version. The help system for version 2.3 answers questions about version 2.3; the help system for version 2.4 answers questions about version 2.4. There is no possibility of a version mismatch between the running software and its documentation.

8.4 Model Size and Acceptance

The most significant practical concern for users and package maintainers is the size of the `help.model` artefact. A Phi-3-Mini model quantised to Q4_K_M in GGUF format occupies approximately 2.1 GB; Qwen2.5-1.5B at the same quantisation level occupies approximately 1.0 GB. These are non-trivial additions to a package.

However, context matters. Professional creative applications such as 3D modelling tools, video editors, and audio production software routinely ship with multi-gigabyte asset packs, sample libraries, and plug-in collections. For such applications, a 1–2 GB addition for a fully local, version-accurate, context-sensitive help system is likely acceptable. For lightweight utilities and command-line tools, the system may be offered as an optional downloadable add-on rather than included in the base package.

The embedding model (~90 MB for `bge-small-en`) and the vector index (size proportional to documentation size; typically 50–200 MB for a well-documented project) add relatively little overhead and are unlikely to be contentious.

9 Discussion

9.1 Telemetry and Privacy

We have described the telemetry log as privacy-preserving by design on the grounds that it is local and never transmitted. This deserves elaboration. The telemetry log does record user behaviour in detail, including what files the user has open and what operations they perform. Even locally stored, this constitutes a privacy-sensitive artefact from the user's perspective.

We recommend the following design principles for FOSS applications implementing this system:

- Telemetry collection is opt-in at first run, with a clear and honest explanation of what is collected, where it is stored, and how it is used.

- The telemetry log file is stored in the user's home directory or application data directory, not a system-wide location, and is readable and deletable by the user.
- The log file format is human-readable (JSON Lines or similar), so users can inspect exactly what has been recorded.
- The application provides a UI to clear the log at any time.
- If the project offers any form of aggregate telemetry submission (e.g., opt-in crash reporting), the interaction context log described here is explicitly excluded from transmission.

9.2 Feedback Loop and Documentation Quality

The telemetry log, aggregated across users with appropriate anonymisation and consent, provides project maintainers with a uniquely valuable signal: a map of where users actually get stuck, and which help queries followed which workflow failures. This is information that is currently invisible to most FOSS projects.

With user consent and a lightweight local aggregation step (computing only non-identifying summary statistics before any voluntary submission), maintainers could learn:

- Which operations generate the most help queries, indicating UX problems or documentation gaps.
- Which retrieved chunks most frequently precede session recovery (the user stopped asking for help and continued working), providing a relevance signal for reranking.
- Which error types are most commonly unresolved by the current help content, identifying topics that require new documentation.

This creates a virtuous cycle in which the help system actively improves documentation quality for future releases, grounded in observed user need rather than maintainer intuition.

9.3 The Context Prioritisation Problem

The folding tape addresses the token budget problem for temporal context, but a related challenge remains: which dimensions of the current application state are most relevant to include alongside the folding tape context in a retrieval query? A complex application may have many simultaneously active state dimensions — the current document type, the active tool, the selection state, the current mode, the contents of active panels — and not all of them are equally predictive of what the user needs to know.

A lightweight learned ranker, trained on (application state, query) pairs from the telemetry logs, could learn to prioritise the state dimensions most predictive of user information need. This is a tractable supervised learning problem given sufficient data, and represents a natural extension to the architecture described here.

9.4 Limitations

The system as described has several limitations that should be acknowledged:

Small model capability ceiling. A 1–3B parameter model, even with strong retrieval grounding, has a lower capability ceiling than a frontier model. For complex, multi-step troubleshooting that requires synthesising information across many disparate documentation sections, the system may fail to produce a helpful answer even when the relevant information is in the index. This is an inherent trade-off of the local, version-locked design.

Documentation quality dependence. The system is only as good as the project's existing documentation. A project with poor docstring coverage, outdated user documentation, or no systematic test suite will produce a sparse and unreliable knowledge base. The system provides strong incentives to improve documentation quality, but it cannot substitute for it.

Initial setup cost. Integrating the telemetry collection layer into an existing application requires instrumentation work. For projects starting from scratch, this is straightforward; for large existing codebases with no telemetry infrastructure, it represents a non-trivial engineering investment.

Cold start. At the beginning of a new session, the folding tape is empty and the system has no contextual advantage over a conventional semantic search. Context quality improves as the session progresses and the tape accumulates events.

10 Conclusion

We have described a system for embedded, context-aware help in FOSS applications that is local, offline, version-locked, and privacy-preserving. The system achieves contextual awareness through structured local telemetry and the folding tape memory model, a hierarchical compression scheme that maintains a complete representation of the user's session history at progressively decreasing resolution as events recede in time.

The folding tape is the central contribution of this paper. It provides a principled solution to the tension between context window efficiency and session history completeness, produces context vectors that are directly useful for retrieval augmentation without requiring the model to infer intent from raw event sequences, and mirrors the structure of human episodic memory in a way that is not coincidental but reflects the natural information structure of the help retrieval task.

The system is practically realisable today using existing open-source components: `llama.cpp` for local inference, `sqlite-vec` or `usearch` for single-file vector indexing, and any of several capable 1–3B SLMs available under open licences. The build-time pipeline integrates naturally into existing CI/CD infrastructure and produces artefacts that are versioned, auditable, and shippable alongside the application binary.

We believe this architecture represents an underexplored but practically important niche: help systems that are not only semantically capable but also epistemically trustworthy —

accurate to the version the user is running, transparent about their knowledge sources, and respectful of the user's privacy. These properties are particularly valuable in the FOSS ecosystem, and we hope this paper encourages their adoption.

Acknowledgements

The author thanks the developers of `llama.cpp`, `sqlite-vec`, and the broader open-source inference and embedding tooling ecosystem, whose work makes the practical realisation of this architecture possible.

References

[1] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474, 2020.

[2] Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. GPTQ: Accurate post-training quantization for generative pre-trained transformers. *arXiv preprint arXiv:2210.17323*, 2022.

[3] Georgi Gerganov et al. llama.cpp: Efficient LLM inference in C/C++. https://github.com/ggerganov/llama.cpp, 2023.

[4] Marah Abdin, Sam Ade Jacobs, Ammar Ahmad Awan, Jyoti Aneja, Ahmed Awadallah, Hany Awadalla, Nguyen Bach, Amit Bahree, Arash Bakhtiari, and others. Phi-3 technical report: A highly capable language model locally on your phone. *arXiv preprint arXiv:2404.14219*, 2024.

[5] Qwen Team. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2025.

[6] Gemma Team, Thomas Mesnard, Cassidy Hardin, Robert Dadashi, Surya Bhupatiraju, Shreya Pathak, Laurent Sifre, Morgane Rivière, Mihir Sanjay Kale, and others. Gemma: Open models based on Gemini research and technology. *arXiv preprint arXiv:2403.08295*, 2024.

[7] Pat Langley. Machine learning for adaptive user interfaces. In *Annual Conference on Artificial Intelligence*, pages 53–62. Springer, 1997.

[8] John R. Anderson, C. Franklin Boyle, and Brian J. Reiser. Intelligent tutoring systems. *Science*, 228(4698):456–462, 1985.

[9] Jeff Wu, Long Ouyang, Daniel M. Ziegler, Nisan Stiennon, Ryan Lowe, Jan Leike, and Paul Christiano. Recursively summarizing books with human feedback. *arXiv preprint arXiv:2109.10862*, 2021.

[10] Yuhuai Wu, Markus Norman Rabe, DeLesley Hutchins, and Christian Szegedy. Memorizing transformers. *arXiv preprint arXiv:2203.08913*, 2022.

[11] Alexis Chevalier, Alexander Wettig, Anirudh Ajith, and Danqi Chen. Adapting language models to compress contexts. *arXiv preprint arXiv:2305.14788*, 2023.

[12] Charles Packer, Vivian Fang, Shishir G. Patil, Kevin Lin, Sarah Wooders, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.

[13] James L. McClelland, Bruce L. McNaughton, and Randall C. O'Reilly. Why there are complementary learning systems in the hippocampus and neocortex: insights from the successes and failures of connectionist models of learning and memory. *Psychological Review*, 102(3):419–457, 1995.

[14] Dharshan Kumaran, Demis Hassabis, and James L. McClelland. What learning systems do intelligent agents need? Complementary learning systems theory updated. *Trends in Cognitive Sciences*, 20(7):512–534, 2016.